The Boundary of Graph Sparsification for Community Detection

Mohammad Dindoost, Bavan DivaaniAazar, Ioannis Koutis, and David A. Bader

Abstract—

Index Terms—Graph sparsification, community detection, modularity, evaluation methodology, graph reduction.

I. Introduction

Community detection is a fundamental task in network analysis [1], [2]. Given a graph, the goal is to partition its vertices into groups that are densely connected internally and sparsely connected to the rest of the graph, and the resulting partitions are used to summarize structure, to identify functional units, and to organize downstream computation. The task is computationally demanding, and the most widely used methods, including modularity optimization [3], [4], [5] and flow-based approaches [6], have running times that scale with the number of edges. On the graphs to which these methods are now routinely applied, that quantity is large [7].

Graph sparsification constructs a subgraph on the same vertex set with substantially fewer edges while preserving properties of the original. The strongest guarantees are spectral. Spielman and Srivastava [8] show that sampling each edge with probability proportional to its effective resistance, and reweighting the retained edges by the inverse of that probability, yields a subgraph with $O(n \log n/\epsilon^2)$ edges whose Laplacian quadratic form approximates that of the original within a factor $1 \pm \epsilon$; cut sparsifiers with comparable guarantees are obtained by related sampling [9], and near-linear-time constructions followed [10], [11]. Because effective resistances are themselves expensive to compute, practical variants approximate them. DSpar [12] replaces the resistance of an edge (u, v) by $1/d_u + 1/d_v$, which bounds it within a factor $2/\alpha$ where α is the normalized spectral gap [13], and thereby obtains a spectral sparsifier at the cost of reading the degree sequence.

Spectral approximation is a statement about the Laplacian, and the eigenvectors of the Laplacian encode cuts and connectivity [14], [15]. Community structure is closely related to those quantities without being identical to them. If the properties preserved by sparsification include community structure, then detection can be performed on the sparsified graph at lower cost and without loss of quality. Whether they do is the question this paper addresses.

The authors are with the Department of Computer Science, New Jersey Institute of Technology, Newark, NJ, USA. E-mail: {md724, bd344, ikoutis, bader}@njit.edu

Satuluri et al. [16] introduced local similarity sparsification, in which each vertex retains the incident edges whose endpoints have the highest Jaccard similarity of neighbourhoods. Evaluating four clustering algorithms on seven networks, they reported speedups commonly between 10× and 50× with little or no loss of cluster quality, and improved clustering accuracy for two of the four algorithms.

The approach was taken up and extended. Hamann et al. [17] conducted a systematic comparison of structure-preserving sparsification methods for social networks and released reference implementations, which are now distributed in standard graph libraries. Wu and Chen [18] trained a generative model to sparsify graphs specifically for community detection, and report that clustering accuracy is retained while as little as five percent of the edges are kept. Sparsification also entered production: the SimClusters system at Twitter [19] prunes its interaction graph before community detection at industrial scale. Blagus et al. [20] report a converse effect, that sampling can promote apparent community structure in social and information networks. Most recently, Chen et al. [21] benchmarked twelve sparsification algorithms against sixteen graph properties on fourteen networks, concluding that no single sparsifier preserves all properties and that the choice should be matched to the downstream task.

The conditions under which the original result was obtained are specific, and they did not travel with it. The two algorithms for which accuracy improved are fixed-k, balance-constrained cut partitioners, whereas the pipeline is now applied to modularity optimizers that choose their own granularity. Accuracy was measured as agreement with external ground-truth communities, whereas quality is now also reported using objectives computed on the graph itself. The speedups were relative to clustering runs requiring hours on graphs of up to 1.2×10^8 edges, and the networks on which the gains were largest have average degrees of 65, 76 and 94, two of them derived from high-throughput assays known to contain false-positive edges; the pipeline is now applied to sparse graphs and to detection algorithms that run in near-linear time. The same paper reports a synthetic sweep in which the method begins to outperform clustering on the unsparsified graph at an average degree of approximately 50. To our knowledge, no subsequent work examines that threshold.

These extensions also change what has to be measured. The question a sparsify-then-detect pipeline is meant to

answer is whether the communities of the original graph can still be recovered after most of its edges are removed. The object of interest is therefore the original graph, and the quantity to be compared is the quality, on that graph, of two partitions: the one obtained by running the detection algorithm on the sparsified graph, and the one obtained by running it on the original graph directly. Both partitions cover the same vertex set, so both can be scored on the same object, and the difference between those two scores is the effect of sparsification.

The sparsifier classes and detection algorithms described above each impose further requirements of the same kind, and several have been noted individually in the literature. Similarity-based sparsifiers [16] retain edges whose endpoints share neighbours, and therefore remove between-community edges preferentially; these are the edges that modularity and conductance penalize, so the sparsifier and the quality measure act on the same quantity. Degree-based sparsifiers [12] select edges by a function of the endpoint degrees alone, and a heavy-tailed degree sequence reproduces several of the resulting effects without any community structure being present, so comparison against a degree-preserving random graph [22] is required to separate the two. Detection algorithms that choose their own granularity [4], [5] return finer partitions on a sparsified graph: Chen et al. [21] document community counts rising by three orders of magnitude across a pruning sweep, and Hamann et al. [17] warn that normalized mutual information is inflated when sparsification fragments the graph, recommending chance-corrected indices in its place. Finally, sparsification is itself a computation, and Gottesbüren et al. [23] observe that its overhead can counteract the speedup it is intended to produce. Each requirement is a property of a specific method class rather than a general caveat, and each determines what a comparison involving that class is entitled to conclude. Section III states the resulting requirements and the control that establishes each one.

Applying these requirements, we re-examined the question across both sparsifier families that have claimed quality gains, degree-based and similarity-based, the latter being the method of Satuluri et al.; four detection algorithms drawn from the two families identified above; seventeen real networks of up to 25 million edges; and a synthetic benchmark in which average degree is swept over an order of magnitude while community structure is held fixed. One arm of the synthetic study uses a fixed-k partitioner, so that the number of communities is identical in the sparsified and unsparsified conditions and granularity is excluded by construction rather than by matching. The study comprises 23 controlled experiments. Predictions and stopping rules were fixed before each experiment was run, and several of our own hypotheses did not survive them.

On the graphs examined in this study, sparsification improves community detection only for detection algorithms that take the number of communities as an input, and only where the graph is dense enough to carry redundant edges. The transition between the two regimes is sharp within a given family of graphs, but its location is not universal. On the synthetic benchmarks we generate it falls near an average degree of 50, which is also the threshold reported in the original synthetic sweep. On the real networks we test it has already occurred at an average degree of 29, the lowest density at which we ran a fixed-k partitioner on a real graph, so the transition there lies at or below that value. We therefore state the condition rather than a threshold, and the numbers we report describe the graphs in this study rather than graphs in general.

Where either condition fails, no benefit of any kind is present. For modularity-family detectors with free granularity, on graphs with average degree between 5 and 33, no sparsifier we tested improves modularity measured on the original graph beyond the variation of a small number of random restarts, at any retention level that preserves quality, under any of the four algorithms examined. Nor is there a speed benefit. Measured end to end, including the cost of the sparsifier itself, the pipeline requires between $0.87\times$ and $1.35\times$ the time of clustering the original graph directly, and at aggressive retention it is slower, because a fragmented graph presents the optimizer with more work rather than less. Every apparent gain we found in this regime, in modularity, in ground-truth recovery, and in the one case where a sparsifier's edge-selection signal is demonstrably structure-aware rather than degree-driven, dissolves when granularity is matched, when chance is subtracted, or when the baseline is given an equal compute budget.

Where both conditions hold, the gain is real. On synthetic graphs with planted communities, holding the number of communities fixed by construction so that no granularity effect is possible, similarity-based sparsification raises both the objective measured on the original graph and chance-corrected agreement with the planted labels. The effect is absent at average degree 11 and 25, present in two thirds of configurations at 49, and present in all of them at 102 and 221. It survives the worst case over ten independent runs of the partitioner, and it is not reproduced by degree-based or uniform sparsification at identical retention, so it is a property of the similarity signal rather than of the edge budget. Injecting spurious edges makes the gain grow monotonically, which is the signature the denoising explanation predicts.

The outcome is therefore determined by two conditions acting together, and neither is sufficient on its own. A fixed-k partitioner applied to a sparse graph shows no gain in any configuration we tested, and a free-granularity optimizer applied to a dense one shows none either once granularity is controlled. The gain appears only where sufficient density and a fixed community count coincide. Which sparsifier is used matters less than either condition, provided its selection signal reflects local structure rather than degree alone. The mechanism is consistent with this. The quantity governing a degree-based sparsifier's effect on modularity is the separation between the scores it assigns to within-community and between-community

edges, and we show by direct intervention that this quantity is controlled by where high-degree-product edges sit relative to community boundaries, at fixed degree sequence, fixed partition and fixed modularity.

Two qualifications attach to this result. Sparsification repairs a weaker detector rather than surpassing a stronger one: a resolution-tuned modularity optimizer on the untouched graph outperforms every sparsified pipeline we measured. And with an exact similarity computation the quality gain is not a speed gain, because the sparsifier costs more than the detection it accelerates.

This paper contributes the following.

- The boundary. A degree threshold near average degree 50 and an algorithm-family split, established under controls that exclude granularity by construction, together with the negative territory delimited across two sparsifier families, four detection algorithms and seventeen networks.
- An evaluation protocol, and the measured cost of omitting it. Each control is motivated by a specific failure it catches, and the consequence of the central omission is quantified at 45 sign reversals in 63 configurations. We also introduce a permuted-weight control for similarity-weighted pipelines, which shows that the benefit attributed to similarity weighting is reproduced, and in one case exceeded, when the weights are randomly permuted across edges.
- A mechanism for the boundary. A decomposition of the score-separation quantity into a sorting term and a granularity term across seventeen networks, and a direct causal test in which that quantity is steered through more than a full sign change while degree sequence, partition, intra-edge fraction and modularity are all held exactly fixed.
- An open phenomenon. In several controlled settings the partitions that best recover reference communities are not the partitions that score best on the objective, and the two criteria move in opposite directions within the same configuration. We report this as an observation rather than a result, and argue that it is the most consequential open question the boundary exposes.

II. Background and Setting

III. What Honest Evaluation Requires

IV. The Mechanism

V. Below the Boundary: Where Sparsification Does Not Help

VI. The Boundary

Everything to this point has been measured on real networks whose average degree lies between 5.5 and 32.6, and on that territory the answer is uniformly negative. That range is not a neutral choice. It is the range in which sparsification for community detection is normally studied, and it lies entirely below the one quantitative boundary the founding work names [16]: a synthetic sweep in which local similarity sparsification "actually outperforms the original clustering starting from degree 50." A negative result confined to one side of a boundary answers half the question. This section supplies the other half.

A. Design

Three properties are required of the setting. Average degree must vary over an order of magnitude while everything else is held fixed. Community labels must be known rather than inferred. And the granularity artifact that accounts for most of the apparent gains in this literature must be removable by construction rather than by control.

Synthetic benchmarks with planted communities supply the first two. A fixed-k partitioner supplies the third. If the number of communities is an input to the algorithm, then the sparsified and unsparsified arms return the same number of communities by definition, and no difference between them can be attributed to one partition being finer than the other.

We generate LFR benchmark graphs [24] with $n = 10^4$ nodes and degree exponent 2.1, matching the generator settings of the original sweep, at nominal average degrees of 10, 25, 50, 100 and 200, at two mixing levels, with three graphs per configuration. Realized degree and mixing are recorded per graph, and all results are reported against realized values. Three sparsifiers are applied at matched realized retention: local similarity sparsification, degree-based sparsification, and uniform random edge deletion. Detection is performed both by a free-granularity modularity optimizer and, through the pymetis bindings, by Metis, a fixed-k balance-constrained cut partitioner from the family the original accuracy claim concerns. Quality is scored on the original graph throughout. Recovery is measured against the planted labels with chance-corrected indices and a size-matched random-partition floor.

B. The threshold

Table I reports, for each average degree, the number of configurations in which similarity-based sparsification improves modularity measured on the original graph, and the number in which it improves chance-corrected agreement with the planted communities, out of twelve configurations per degree.

Below average degree 25, the method is harmful on both measures in every configuration tested. At 49.4 it becomes beneficial in two thirds of them. By 101.9 it is beneficial in all twelve configurations on recovery and ten of twelve on the objective. The strongest individual cells occur at aggressive retention: at average degree 49.4 and 15% retention, the partition obtained from the sparsified graph scores +0.005 modularity and +0.152 AMI above the partition obtained from the original graph, and at 220.9 the modularity gain reaches +0.012.

The location of this transition was not chosen by us, and it was not known to us when the experiment was designed. It coincides with the degree at which the original

TABLE I
Fixed-k partitioner (Metis), realized mixing ≈ 0.6. Counts are configurations out of twelve in which similarity sparsification improves the quantity named, with the mean change beside them. Modularity is measured on the original graph. Because k is an input to the partitioner, the sparsified and unsparsified arms return identical community counts, so no granularity effect is possible.

Avg. degree	$\Delta Q > 0$	mean ΔQ	ΔAMI > 0	mean ΔAMI
11.2	0/12	−0.103	0/12	−0.216
24.6	0/12	−0.046	6/12	−0.072
49.4	8/12	−0.016	9/12	+0.042
101.9	10/12	+0.006	12/12	+0.116
220.9	12/12	+0.009	12/12	+0.093

authors reported their method beginning to overtake the unsparsified baseline.

C. Three ways the result could be spurious

a) Run-to-run variation: Metis is not deterministic across option seeds. We repeated the decisive cells with ten partitioner seeds per graph and compared the worst sparsified run against the best unsparsified run. The gain survives that comparison at every degree at or above 49.4, with worst-case modularity differences of +0.001, +0.005 and +0.008, and worst-case AMI differences of +0.134, +0.102 and +0.064. Seed standard deviations are 0.002 in modularity and 0.011 in AMI, so the effect exceeds the noise it must clear by a factor of between two and sixty.

b) The edge budget rather than the selection: Any sparsifier at 15% retention produces a smaller graph that is denser within communities, and a fixed-k partitioner may simply behave better on such a graph. Degree-based and uniform random sparsification, applied to the same graphs at the same realized retention, do not reproduce the effect at any degree. Their modularity changes run from −0.145 to −0.003, and their recovery changes from −0.480 to +0.032. What produces the gain is the similarity signal used to choose which edges to keep, not the number of edges kept.

c) Granularity: The partitioner takes the community count as an input, and both arms are run with the same value, so the two partitions being compared have identical community counts by construction. This is the artifact that no amount of matching fully excludes on a free-granularity detector, and here it is excluded structurally. We also record cluster-size balance and find it slightly improved rather than degraded under sparsification, so the gain is not purchased by producing more uneven communities.

D. The same graphs, a different algorithm family

Running a free-granularity modularity optimizer on the identical graphs gives a different answer. Modularity gains never exceed the spread of a handful of random restarts. Recovery gains do appear, and they grow with degree, reaching +0.096 AMI, but they do not survive the granularity control. Partitioning the original graph at a resolution tuned to match the sparsified partition's community count recovers the planted communities as well or better in twenty-one of the twenty-nine configurations where the comparison is defined. What appeared to be a benefit of sparsification is the benefit of a finer partition, obtainable without touching the graph.

The variable that decides the outcome is therefore the interaction of density with the detection algorithm's degrees of freedom, not the sparsifier and not the graph alone. A partitioner constrained to return a fixed number of balanced groups is helped, above a density threshold, by having its misleading edges removed. An optimizer free to choose its own granularity already has a cheaper route to the same improvement, and takes it.

E. Why density matters: the denoising account

The original authors attributed their gains to the removal of spurious edges, noting that their strongest results came from an assay network known to contain many false-positive interactions. That explanation makes a testable prediction. If sparsification helps by deleting edges that do not belong to the generative structure, then its benefit should grow with the proportion of such edges.

We inject uniformly random edges into benchmark graphs while holding the planted labels fixed, and measure recovery against those labels. The prediction holds. Recovery gain rises monotonically with the injected fraction in every tested configuration for similarity-based sparsification, with rank correlations of unity between noise level and gain. At average degree 50 and 20% retention, the gain grows from +0.062 with no injected noise to +0.184 when the number of spurious edges equals the number of real ones. Degree-based sparsification shows no such response, which is consistent with its selection signal being a function of degree rather than of local structure.

Two qualifications apply. The gain does not vanish when no noise is injected, so denoising adds to an effect that is already present rather than being its sole cause. And a benchmark graph's noise is random by construction, whereas the spurious edges produced by a real measurement process need not be. The experiment establishes the direction of the mechanism, not its magnitude in the field.

F. What the positive result does not say

Two facts constrain how this finding should be read, and both come from the same experiments.

Sparsification repairs a weaker detector rather than producing the best available partition. At every degree tested, a resolution-tuned modularity optimizer running on the untouched graph recovers the planted communities better than any sparsified pipeline. At average degree 49.4 it reaches 0.956 AMI, against 0.754 for the sparsified fixed-k pipeline. Sparsification lifts the fixed-k partitioner from 0.596 to 0.754, a real and substantial improvement to that algorithm, and still leaves it behind an algorithm that never needed sparsification. The defensible statement is

that similarity sparsification repairs a fixed-k cut partitioner on dense graphs, not that it improves community detection.

In our implementation the quality gain is also not a speed gain. Computing exact pairwise similarity costs more than it saves. End to end, the pipeline runs at 1.49× the baseline at average degree 10, 1.01× at 100, and 0.58× at 200, which is slower than not sparsifying at all, even though detection alone accelerates by up to 7.2×. The original speedups were obtained with an approximate similarity computation reported as two orders of magnitude cheaper, measured against clustering runs that took hours. Both cost bases differ from ours by enough to account for the discrepancy without any disagreement about what should be counted.

One part of the original report does not reproduce. At high mixing, where the planted structure is nearly absent and baseline recovery falls to between 0.03 and 0.19, we find no gain under either detector at any degree, whereas the original sweep reports its largest improvement in that regime. Whether this is a genuine non-reproduction or a floor effect of the metrics we use cannot be settled at that mixing level, and we report it unresolved.

VII. Discussion

VIII. Conclusion

References

[1] S. Fortunato, "Community detection in graphs," Physics Reports, vol. 486, no. 3–5, pp. 75–174, 2010.

[2] S. Fortunato and M. E. Newman, "20 years of network community detection," Nature Physics, vol. 18, no. 8, pp. 848–850, 2022.

[3] M. E. J. Newman and M. Girvan, "Finding and evaluating community structure in networks," Physical Review E, vol. 69, no. 2, p. 026113, 2004.

[4] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," Journal of Statistical Mechanics: Theory and Experiment, vol. 2008, no. 10, p. P10008, 2008.

[5] V. A. Traag, L. Waltman, and N. J. Van Eck, "From Louvain to Leiden: Guaranteeing well-connected communities," Scientific Reports, vol. 9, no. 1, pp. 1–12, 2019.

[6] M. Rosvall and C. T. Bergstrom, "Maps of random walks on complex networks reveal community structure," Proceedings of the national academy of sciences, vol. 105, no. 4, pp. 1118–1123, 2008.

[7] J. Leskovec, K. J. Lang, A. Dasgupta, and M. W. Mahoney, "Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters," Internet Mathematics, vol. 6, no. 1, pp. 29–123, 2009.

[8] D. A. Spielman and N. Srivastava, "Graph sparsification by effective resistances," in Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing, 2008, pp. 563–568.

[9] A. A. Benczúr and D. R. Karger, "Randomized approximation schemes for cuts and flows in capacitated graphs," SIAM Journal on Computing, vol. 44, no. 2, pp. 290–319, 2015.

[10] D. A. Spielman and S.-H. Teng, "Nearly-linear time algorithms for graph partitioning, graph sparsification, and solving linear systems," in Proceedings of the Thirty-Sixth Annual ACM Symposium on Theory of Computing, 2004, pp. 81–90.

[11] ——, "Nearly linear time algorithms for preconditioning and solving symmetric, diagonally dominant linear systems," SIAM Journal on Matrix Analysis and Applications, vol. 35, no. 3, pp. 835–885, 2014.

[12] Z. Liu, K. Zhou, Z. Jiang, L. Li, R. Chen, S.-H. Choi, and X. Hu, "Dspar: An embarrassingly simple strategy for efficient GNN training and inference via degree-based sparsification," Transactions on Machine Learning Research, 2023.

[13] L. Lovász, "Random walks on graphs," Combinatorics, Paul Erdős Is Eighty, vol. 2, no. 1–46, p. 4, 1993.

[14] U. von Luxburg, "A tutorial on spectral clustering," Statistics and Computing, vol. 17, no. 4, pp. 395–416, 2007.

[15] J. Shi and J. Malik, "Normalized cuts and image segmentation," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 22, no. 8, pp. 888–905, 2000.

[16] V. Satuluri, S. Parthasarathy, and Y. Ruan, "Local graph sparsification for scalable clustering," in Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data, 2011, pp. 721–732.

[17] M. Hamann, G. Lindner, H. Meyerhenke, C. L. Staudt, and D. Wagner, "Structure-preserving sparsification methods for social networks," Social Network Analysis and Mining, vol. 6, no. 1, p. 22, 2016.

[18] H.-Y. Wu and Y.-L. Chen, "Graph sparsification with generative adversarial network," in 2020 IEEE International Conference on Data Mining (ICDM), 2020.

[19] V. Satuluri, Y. Wu, X. Zheng, Y. Qian, B. Wichers, Q. Dai, G. M. Tang, J. Jiang, and J. Lin, "SimClusters: Community-based representations for heterogeneous recommendations at Twitter," in Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD), 2020, pp. 3183–3193.

[20] N. Blagus, L. Šubelj, G. Weiss, and M. Bajec, "Sampling promotes community structure in social and information networks," Physica A: Statistical Mechanics and its Applications, vol. 432, pp. 206–215, 2015.

[21] Y. Chen, H. Ye, S. Vedula, A. Bronstein, R. Dreslinski, T. Mudge, and N. Talati, "Demystifying graph sparsification algorithms in graph properties preservation," Proceedings of the VLDB Endowment, vol. 17, no. 3, pp. 427–440, 2024.

[22] M. Molloy and B. Reed, "A critical point for random graphs with a given degree sequence," Random Structures & Algorithms, vol. 6, no. 2–3, pp. 161–180, 1995.

[23] L. Gottesbüren, N. Maas, D. Rosch, P. Sanders, and D. Seemaier, "Linear-time multilevel graph partitioning via edge sparsification," in 33rd Annual European Symposium on Algorithms (ESA), 2025, arXiv:2504.17615.

[24] A. Lancichinetti, S. Fortunato, and F. Radicchi, "Benchmark graphs for testing community detection algorithms," Physical Review E, vol. 78, no. 4, p. 046110, 2008.